Linux Fundamentals — Day 1

Beginner-friendly study notes · Day 1

Linux Basics and Core Commands

Beginner-friendly notes covering Linux fundamentals, file-system navigation, file and directory management, permissions, and basic user administration.

Environment: Examples use Kali Linux or a Debian-based distribution. Some paths and log files differ between distributions. Run commands only on systems you own or are authorized to administer.

Table of Contents

1. [Introduction to Linux](#1-introduction-to-linux)
2. [Linux Distributions](#2-linux-distributions)
3. [GUI and CLI](#3-gui-and-cli)
4. [Basic Linux Terminology](#4-basic-linux-terminology)
5. [Explore the Terminal](#5-explore-the-terminal)
6. [Linux File-System Structure](#6-linux-file-system-structure)
7. [Paths: Absolute and Relative](#7-paths-absolute-and-relative)
8. [User Authentication Files](#8-user-authentication-files)
9. [Navigation Commands](#9-navigation-commands)
10. [Search, Pipes, and Command Chaining](#10-search-pipes-and-command-chaining)
11. [File and Directory Management](#11-file-and-directory-management)
12. [Viewing and Editing Files](#12-viewing-and-editing-files)
13. [Linux Permissions](#13-linux-permissions)
14. [Ownership and `sudo`](#14-ownership-and-sudo)
15. [Hands-on Labs](#15-hands-on-labs)
16. [Review Questions](#16-review-questions)

1. Introduction to Linux

Linux is an open-source operating-system kernel first released by Linus Torvalds in 1991. A complete Linux operating system typically combines the kernel with system utilities, libraries, package-management tools, and applications.

Linux is widely used for servers, cloud infrastructure, embedded systems, software development, and cybersecurity. Security professionals benefit from learning Linux because many servers and security tools use it, and the command line is central to system administration, automation, and investigation.

Common uses

- **Servers and cloud:** hosting websites, applications, and services.
- **Cybersecurity:** security testing, digital forensics, monitoring, and incident response.
- **DevOps:** containers, deployment pipelines, and infrastructure automation.
- **Networking:** configuring and troubleshooting network services.
- **Development and research:** programming, data processing, and scientific computing.

Linux and Windows: a general comparison

Feature	Linux	Windows
Licensing	Many distributions are free and open source	Common desktop editions use proprietary licensing
Source code	Core components are often available for inspection and modification	Most core source code is closed
User interface	Command line and graphical desktops are available	Graphical desktop is prominent; command-line tools are also available
Software installation	Often uses package managers such as `apt` or `dnf`	Commonly uses installers, Microsoft Store, and package-management tools
Typical environments	Servers, cloud, development, embedded systems, security	Desktop computing, enterprise environments, gaming, and servers
Security	Provides detailed permissions and security controls; configuration still matters	Provides built-in security controls; configuration still matters

No operating system is automatically secure. Security depends on updates, configuration, access controls, software, and operational practices.

2. Linux Distributions

A **Linux distribution** is an operating-system package built around the Linux kernel and supplied with utilities, software, and a package-management system.

Distribution	Common purpose
Ubuntu	Desktop use, development, and servers
Kali Linux	Penetration testing and security research
Debian	Stable general-purpose systems and servers
Red Hat Enterprise Linux (RHEL)	Enterprise environments and commercial support
Parrot OS	Security, privacy, and penetration-testing workflows

Choose a distribution based on the task, hardware, support requirements, and software availability.

3. GUI and CLI

GUI — Graphical User Interface

A GUI lets a person interact with a system using windows, icons, menus, and buttons.

Advantages

- Easy to start using.
- Visual and discoverable.
- Useful for desktop applications.

Limitations

- Repetitive operations may take longer.
- Automation can be less convenient for some tasks.
- Graphical applications may consume additional resources.

CLI — Command-Line Interface

A CLI accepts typed commands in a terminal.

Advantages

- Efficient for repetitive tasks.
- Supports scripting and automation.
- Useful on remote servers.
- Provides detailed control over system tools.

Limitations

- Commands and their options must be learned.
- Incorrect commands can modify or delete data.

Examples:

```
# GUI: open a file manager and click a folder
# CLI: change to a directory
cd /home/kali/Documents
```

Both interfaces are useful. Security professionals commonly use the CLI for administration, automation, and analysis.

4. Basic Linux Terminology

Term	Meaning
Kernel	Core component that manages hardware and system resources
Shell	Program that interprets commands, such as Bash or Zsh
Terminal	Application or interface through which a shell is accessed
Root user	Administrative account, normally with user ID 0
Directory	A filesystem object used to organize files and other directories
Path	The location of a file or directory
Package	A software bundle managed by a package system
Daemon/service	A background process that provides a system or network function

5. Explore the Terminal

Open a terminal and try these commands:

```
whoami       # Print the current username
uname -a     # Show kernel and system information
hostname     # Show the system's hostname
date         # Display the current date and time
```

These commands are read-only and are useful for quickly identifying the current account and system.

6. Linux File-System Structure

Linux uses a single directory hierarchy beginning at `/`, called the **root directory**. Other directories appear below it. This differs from the drive-letter layout commonly seen in Windows.

```
/
■■■ bin/       Essential command binaries (may link to /usr/bin)
■■■ boot/      Bootloader and kernel-related files
■■■ dev/       Device interfaces
```

```
■■■ etc/       System-wide configuration
■■■ home/      Regular users' home directories
■■■ proc/      Virtual information about processes and the kernel
■■■ root/      Home directory for the root account
■■■ tmp/       Temporary files
■■■ usr/       Many applications, libraries, and utilities
■■■ var/       Variable data, including logs and caches
```

The exact layout can vary between distributions. Some directories may be symbolic links, and permissions determine which users can access their contents.

Why these directories matter

- `/etc`: system and service configuration.
- `/home`: personal directories for regular users.
- `/var/log`: many system and application logs.
- `/tmp`: temporary data; access and cleanup behavior depend on system configuration.
- `/proc`: virtual files exposing process and kernel information.
- `/dev`: interfaces to devices and other system resources.
- `/root`: home directory for the root account.

Root directory versus root user

These terms are different:

- `/` is the top of the filesystem hierarchy.
- `root` is the administrative user account.
- `/root` is the root user's home directory.

Files and directories

Feature	File	Directory
Typical role	Stores data or program content	Organizes filesystem entries
Examples	`notes.txt`, `script.sh`	`Documents/`, `reports/`
Common creation command	`touch` for an empty file	`mkdir`

7. Paths: Absolute and Relative

An **absolute path** begins at `/` and specifies a location from the filesystem root. A **relative path** is interpreted from the current working directory.

Path expression	Meaning
`/home/kali/Documents/notes.txt`	Absolute path
`Documents/notes.txt`	Relative path
`.`	Current directory
`..`	Parent directory
`./script.sh`	A script in the current directory
`~`	The current user's home directory

Examples:

```
pwd                         # Print current working directory
cd /home/kali/Documents     # Navigate using an absolute path
cd Documents                # Navigate using a relative path, if present
cd ..                       # Move to the parent directory
cd ~                        # Move to the current user's home directory
cd -                        # Return to the previous working directory
```

Use `pwd` when unsure where you are before running commands that change files.

8. User Authentication Files

Two important Linux files are `/etc/passwd` and `/etc/shadow`.

`/etc/passwd`

This file contains account information such as the username, user ID (UID), group ID (GID), descriptive field, home directory, and login shell. It is normally readable by regular users, but it does **not** normally store password hashes.

Example entry:

```
kali:x:1000:1000:Kali:/home/kali:/bin/bash
```

The `x` in the password field indicates that password information is stored separately on systems using the shadow-password scheme.

```
cat /etc/passwd
```

`/etc/shadow`

This file stores password-aging information and password-hash fields on systems using shadow passwords. Access is restricted because this data is sensitive.

```
sudo cat /etc/shadow
```

Use this command only on a system you administer and only when there is a legitimate need. Password hashes should be protected because stolen hashes may be subject to offline guessing attacks.

9. Navigation Commands

`pwd` — show the current location

```
pwd
```

`ls` — list directory contents

```
ls          # Basic listing
ls -l       # Detailed listing
ls -a       # Include hidden entries
ls -la      # Detailed listing, including hidden entries
ls -lh      # Display file sizes in a human-readable format
ls /etc     # List a particular directory without changing into it
```

Names beginning with `.` are treated as hidden by common Linux listing conventions.

`cd` — change directory

```
cd /etc         # Navigate to /etc
cd ..           # Move up one directory
cd ~            # Go to your home directory
cd -            # Return to the previous directory
```

`tree` — display directory structure

```
tree            # Show the current directory tree
tree /etc       # Show the /etc tree
tree -L 2       # Limit display depth to two levels
```

If `tree` is not installed on a Debian-based system, install it with:

```
sudo apt update
sudo apt install tree
```

`clear` — clear the terminal display

```
clear
```

`Ctrl + L` is also commonly used to clear the visible terminal screen.

10. Search, Pipes, and Command Chaining

`grep` — search text

`grep` searches file contents or command output for matching patterns.

```
grep 'root' /etc/passwd            # Find lines containing root
grep -i 'failed' auth.log          # Ignore letter case
grep -n 'error' app.log            # Include line numbers
grep -r 'password' /etc/            # Search files recursively (permissions may limit results)
grep -v 'root' users.txt            # Show lines that do not match root
grep -c 'failed' auth.log          # Count matching lines
grep '^root' /etc/passwd           # Lines beginning with root
grep 'bash$' /etc/passwd           # Lines ending with bash
```

File paths and available logs differ by distribution. For example, authentication events may be recorded in different files or in the system journal.

`find` — locate files and directories

```
find /home -name '*.txt'           # Find .txt files under /home
find /tmp -type f                  # Find regular files under /tmp
find / -name 'passwd' 2>/dev/null  # Search from /; hide permission errors
find / -type f -perm -4000 2>/dev/null
```

The last example searches for regular files with the SUID permission bit set. Review such results carefully: SUID is sometimes legitimate, but unexpected or vulnerable SUID programs can create security risks. Searching the entire filesystem may take time and produce permission errors.

Pipes (`|`)

A pipe sends one command's standard output to another command's standard input.

```
cat /etc/passwd | wc -l
grep 'Failed' auth.log | wc -l
ps aux | grep ssh
ls -la | sort
cat /etc/passwd | cut -d: -f1 | sort
```

For example:

```
grep 'Failed' auth.log | wc -l
```

This counts lines containing `Failed` in the specified log file. It counts matching lines, not necessarily unique login attempts.

A pipeline can combine multiple small tools:

```
grep 'Failed' auth.log | sort | uniq -c
```

The result groups identical matching lines. To summarize events by source IP, first inspect the actual log format and select the correct field; field positions vary between systems and log formats.

`mkdir` — create directories

```
mkdir project                # Create one directory
mkdir dir1 dir2 dir3         # Create several directories
mkdir -p work/reports/2026   # Create parent directories as needed
mkdir .config                # Create a hidden directory
ls -la                       # Include hidden entries in the listing
```

11. File and Directory Management

`touch` — create an empty file or update timestamps

```
touch notes.txt
touch file1.txt file2.txt
```

`touch` can also update timestamps. Existing file content is not erased by a normal `touch` operation.

`cp` — copy

```
cp file.txt /tmp/file.txt       # Copy a file
cp -r mydir /tmp/               # Copy a directory recursively
cp -p file.txt copy.txt         # Preserve selected metadata where possible
```

Make sure the destination is correct before copying sensitive files.

`mv` — move or rename

```
mv file.txt Documents/          # Move a file into a directory
mv oldname.txt newname.txt      # Rename a file
mv mydir /tmp/                  # Move a directory
```

`rm` — remove

```
rm file.txt                     # Remove a file
rm -i file.txt                  # Ask before removing a file
rm -r mydir/                    # Remove a directory and its contents recursively
```

Caution: `rm` normally bypasses a graphical recycle bin. Recursive and force options can delete many files without an interactive confirmation. Verify the path before executing a deletion command. Avoid testing destructive commands on important data.

File extensions

Linux does not depend solely on filename extensions to determine a file's type, but extensions are useful labels.

Extension	Common use
`.txt`	Plain text
`.pdf`	PDF document
`.jpg`, `.png`	Images
`.sh`	Shell script
`.py`	Python script
`.html`	HTML document
`.log`	Log data
`.conf`	Configuration file

SUID files

The Set User ID (SUID) permission bit can cause an executable to run with the file owner's effective user ID. Some SUID programs are required by the system. Unexpected SUID files or unsafe programs deserve investigation during a security review.

```
find / -type f -perm -4000 2>/dev/null
```

Run security checks only on systems you own or are authorized to assess.

12. Viewing and Editing Files

```
cat /etc/passwd              # Print a file
less /var/log/syslog         # Read a file page by page; press q to quit
head -n 20 file.log          # Show the first 20 lines
tail -n 20 file.log          # Show the last 20 lines
tail -f file.log             # Follow new lines as they are appended
nano notes.txt               # Open or create a text file in Nano
```

Use `Ctrl + C` to stop a running `tail -f` session.

Log filenames vary by distribution. If a path such as `/var/log/syslog` does not exist, check the distribution's logging configuration and system journal.

13. Linux Permissions

Linux permissions control which users may read, modify, or execute files and which users may access directories.

A long listing might contain:

```
-rwxr-xr-- 1 root security 4096 Jan 01 file.sh
```

The first character indicates the object type (`-` for a regular file, `d` for a directory, and `l` for a symbolic link). The next nine characters represent permissions for the owner, group, and others.

For `-rwxr-xr--`:

- **Owner (`rwx`):** can read, write, and execute.
- **Group (`r-x`):** can read and execute, but not write.
- **Others (`r--`):** can read, but not write or execute.

Permission values

Permission	Symbol	Numeric value	General meaning
Read	`r`	4	Read file contents; for directories, list entries subject to other permissions
Write	`w`	2	Modify a file; for directories, create or remove entries subject to other permissions
Execute	`x`	1	Execute a file; for directories, traverse/access entries subject to other permissions
None	`-`	0	No permission of that type

Numeric permissions add the values for each class:

- 7 = read + write + execute (`rwx`)
- 6 = read + write (`rw-`)
- 5 = read + execute (`r-x`)
- 4 = read only (`r--`)

`chmod` — change permissions

Numeric (octal) mode

```
chmod 755 script.sh       # Owner rwx; group and others r-x
chmod 644 config.txt      # Owner rw-; group and others r--
chmod 600 id_rsa          # Owner rw-; no access for group or others
chmod 700 private.sh      # Owner has full access only
chmod 777 public.sh       # Everyone has full access; usually unsafe
```

Typical interpretations:

Mode	Permission string	Common use
600	rw-------	Private files such as SSH private keys
644	rw-r--r--	Files that should be writable by the owner and readable by others
700	rwx------	Private executable scripts or directories
755	rwxr-xr-x	Programs or directories accessible to others for reading/traversal
777	rwxrwxrwx	Grants all permission types to everyone; generally avoid

Choose permissions according to the actual access requirement. The correct mode depends on the file and its use.

Symbolic mode

```
chmod +x script.sh       # Add execute permission for user, group, and others
chmod u+w file.txt       # Add write permission for the owner
chmod g-w file.txt       # Remove write permission from the group
chmod o-rwx secret.txt   # Remove all permissions from others
chmod -R 755 mydir/      # Apply recursively; use caution
```

Recursive permission changes can unintentionally expose or break files. Review the target directory before using `-R`.

14. Ownership and `sudo`

`chown` and `chgrp`

Ownership determines which user and group are associated with a filesystem object.

```
sudo chown kali file.txt
sudo chown kali:sudo file.txt
sudo chgrp security reports/
```

Change ownership recursively only when the entire directory tree should have the same ownership:

```
sudo chown -R www-data /var/www/
```

`sudo`

`sudo` lets an authorized user run a command with elevated privileges, commonly as root. It is preferable to routinely using an unrestricted root session because it supports controlled access and auditing.

```
sudo apt update              # Refresh package metadata
sudo -l                      # Show permitted sudo commands
sudo -u alice command        # Run a command as user alice, if permitted
sudo -i                      # Start a root login shell, if authorized
```

Some shell configurations also support `sudo !!` to repeat the previous command with `sudo`; this is shell-dependent and should be used only after checking the command.

Security practices

- Grant administrative access only to trusted users.
- Do not share the root password.
- Review rules in `/etc/sudoers` and files under `/etc/sudoers.d/` carefully; use `visudo` to edit sudo policy safely.
- Keep an audit trail and review system logs according to the distribution's logging configuration.
- Do not give untrusted users unrestricted administrative access.

15. Hands-on Labs

Complete these exercises in your own Linux virtual machine or another system where you have permission to work.

Lab A — Basic system information

1. Open a terminal.
2. Run `whoami`, `uname -a`, `hostname`, and `date`.
3. Write down what each command reports.
4. Run `pwd` to identify your current directory.

Lab B — Navigate the filesystem

```
pwd
ls -la
cd /etc
ls
cd ~
```

Optional, if `tree` is installed:

```
tree -L 2 /etc
```

Identify hidden entries and observe how absolute and relative paths differ.

Lab C — Create and manage files

```
mkdir -p ~/labwork/reports/2026
cd ~/labwork/reports/2026
touch notes1.txt notes2.txt notes3.txt
ls -la
cp notes1.txt /tmp/
mv notes2.txt renamed-notes.txt
```

Inspect the results. Remove only files you created for this lab and no longer need.

Lab D — Search text and files

```
grep "$(whoami)" /etc/passwd
find ~ -name '*.txt'
```

Try searching for a word in one of your own text files:

```
echo "Linux command practice" > practice.txt
grep -n 'Linux' practice.txt
```

Lab E — Inspect and change permissions

```
cd ~/labwork/reports/2026
touch secret.txt
ls -l secret.txt
chmod 600 secret.txt
ls -l secret.txt
```

Create a simple script:

```
printf '#!/bin/bash\necho "Hello from Linux"\n' > myscript.sh
chmod +x myscript.sh
./myscript.sh
```

Check the resulting permissions and explain why an executable script needs execute permission.

Lab F — Review administrative access

```
sudo -l
```

Review the output. Do not attempt to bypass access controls or run commands requiring privileges you have not been authorized to use.

16. Review Questions

Module 1 — Linux fundamentals

1. What is Linux, and who began developing its kernel?
2. Give three differences between Linux and Windows.
3. How do GUI and CLI differ? Why is the CLI useful in cybersecurity?
4. What is a Linux distribution? Name three distributions and their common purposes.
5. What does the kernel do?

Module 2 — Filesystem and navigation

6. What is `/`, and how does Linux's filesystem layout differ from Windows drive letters?
7. Describe the purpose of `/etc`, `/home`, `/var/log`, and `/tmp`.
8. Explain the difference between absolute and relative paths.
9. What does `ls -la` show that `ls` may not show?
10. Why should temporary directories and their contents be handled carefully?
11. What information is stored in `/etc/passwd` and `/etc/shadow`?
12. How does a pipe (`|`) connect commands?

Module 3 — File management

13. Compare `rm file.txt` with `rm -r directory/`.
14. Which command can locate `.txt` files under your home directory?
15. What does `tail -f` do, and when is it useful?
16. How would you search a configuration directory for the word `password`?
17. What is the SUID permission bit, and why should unexpected SUID programs be reviewed?

Module 4 — Permissions and users

18. Interpret the permission string `-rwxr-xr--`.
19. Which `chmod` mode gives the owner read/write access, the group read-only access, and others no access?
20. Why can `chmod 777` be unsafe?
21. How do `chown` and `chmod` differ?
22. What is `sudo`, and why should administrative access be restricted?

Quick Command Reference

Command	Purpose
`whoami`	Display the current username
`uname -a`	Display system and kernel information
`pwd`	Print the current directory
`ls -la`	List entries, including hidden ones, in long format
`cd PATH`	Change directory
`mkdir -p PATH`	Create a directory hierarchy
`touch FILE`	Create an empty file or update timestamps
`cp SOURCE DEST`	Copy files
`mv SOURCE DEST`	Move or rename files
`rm FILE`	Remove a file
`cat FILE`	Print file contents
`less FILE`	Read a file interactively
`head FILE`	Show the beginning of a file
`tail FILE`	Show the end of a file
`grep PATTERN FILE`	Search text
`find PATH ...`	Search filesystem entries
`chmod MODE FILE`	Change permissions
`chown USER FILE`	Change ownership
`sudo COMMAND`	Run a permitted command with elevated privileges

Notes

- Commands, package names, log locations, and default permissions may vary by distribution.
- Use `man COMMAND` (for example, `man chmod`) to read a command's manual page.
- Practise in a virtual machine and keep backups before making system-level changes.
- Perform security testing only on systems you own or have explicit permission to assess.